An Absorbing Set Without Gallager-B Trapping

October 26, 2026

Abstract

We give a seven-variable $(7, 0)$-absorbing set for which every binary error support is accepted by the zero-initialized syndrome-based Gallager-B decoder at initialization or after one iteration. Both the hard-decision syndrome and the parity of outgoing extrinsic messages match the input syndrome. Thus no subset produces the persistent syndrome mismatch or nonconvergence in the trapping-set definition of Morris, Pllaha and Kelley. The matrix consists of all seven nonzero simplex checks: majority voting recovers the single-column representative of each Hamming syndrome. To distinguish syndrome acceptance from quantum recovery, we also give an explicit fourteen-qubit CSS pair in which this absorbing component corrects every error in the decoded binary sector modulo the entire opposite stabilizer span. A nontrivial logical operator remains on the other component. Complete finite proofs and the stopped-execution argument are checked in Lean, with an independent literal-matrix verifier.

1 Question and answer

Morris, Pllaha and Kelley ask in Section VI of *Absorbing Sets in Quantum LDPC Codes*, arXiv:2307.14532v2, May 10, 2024, whether every absorbing set is a trapping set for their syndrome-based Gallager-B decoder [1]. Their Definition 5 requires each selected variable to have strictly more neighbors of even induced degree than of odd induced degree. Definition 3 calls a nonempty set trapping if some error supported on it produces a neighboring check that is not eventually correct or a selected variable that does not eventually converge.

Theorem 1. *There is a connected seven-variable* $(7, 0)$*-absorbing Tanner graph such that every error supported on its variables has an accepted, constant terminal output after at most one full Gallager-B iteration. Consequently the variable set is not trapping in the sense of Definition 3 of [1]. Moreover it occurs as a component of an explicit CSS code with a nontrivial logical operator, in which every error in this binary sector supported on the component is corrected modulo stabilizers.*

The first assertion is proved in Section 3; the CSS assertion is proved in Section 4. The precise source-to-model comparison is in Section 5. Appendix A contains central Lean definitions, complete proof bodies and direct source links.

The example uses redundant checks, check degree four, variable degree four and Tanner four-cycles. None is forbidden in the cited general question. The selected component is connected; the fourteen-qubit ambient graph has two components. We do not assert a connected-ambient, girth-constrained or column-weight-three counterexample. We also do not assert correction of both Pauli sectors on the selected component, or of arbitrary errors on the other component.

2 The actual matrix

Index checks and variables by the nonzero vectors of $\mathbb{F}_2^3$, in the integer order $1, \dots, 7$ with least significant binary digit first. Let $H_{a,v} = a \cdot v$. Explicitly,

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}. \tag{1}$$

Write A for all seven variables. Every row and column of H has weight four, so all neighboring checks have even induced degree and every variable has four even and no odd neighbors. Hence A is $(7, 0)$-absorbing. Any pair of distinct variables has two common check neighbors: two independent linear equations $a \cdot u = a \cdot v = 1$ have two solutions in $\mathbb{F}_2^3$. In particular the Tanner graph is connected and has four-cycles. The finite statements `degrees`, `full_set_absorbing` and `first_block_connected` check the actual matrix, not an abstract replacement graph.

3 One iteration accepts every supported error

All arithmetic in check messages is in $\mathbb{F}_2$. For an arbitrary input error $e \in \mathbb{F}_2^7$ put

$$p = \sum_{v \neq 0} e_v v \in \mathbb{F}_2^3, \qquad s = He^{\mathsf{T}}.$$

Then $s_a = a \cdot p$. Zero initial variable-to-check messages imply that check a sends s_a to each of its neighbors. A hard decision is the strict majority of all incoming check messages, with ties resolved to zero. The outgoing message toward a takes the same strict majority after excluding a.

Lemma 2. *For $v \neq 0$, the number of neighboring checks sending one initially is zero if $p = 0$, four if $p = v$, and two otherwise. Thus the first hard output is*

$$\hat{e} = 0 \quad (p = 0), \qquad \hat{e} = \mathbf{e}_p \quad (p \neq 0),$$

where $\mathbf{e}_p$ is the singleton indicator at column p. In every case $H\hat{e}^{\mathsf{T}} = s$.

Proof. The vote count is the number of $a \in \mathbb{F}_2^3 \setminus \{0\}$ satisfying $a \cdot v = a \cdot p = 1$. If $p = 0$ there are none. If $p = v$ there are four. Otherwise the two nonzero, distinct vectors v, p are independent and the two independent equations have two solutions. The strict majority of four bits is one only in the second case. The asserted syndrome follows from $H\mathbf{e}_p^{\mathsf{T}} = (a \cdot p)_a$. All counts and the displayed output formula have matching kernel-checked finite Lean proofs, including all 128 inputs. □

The conclusion also survives the alternate syndrome calculation from outgoing extrinsic variable messages, which is relevant to some computations in [1].

Lemma 3. *Let $x'_{v \to a}$ be the first outgoing extrinsic message. Then $\sum_{v : a \cdot v = 1} x'_{v \to a} = s_a$ for every check a.*

Proof. For $p = 0$ all messages vanish. For $p \neq 0$, variable p sends one on all four incident edges. At $v \neq p$ there were two ones and two zeros before excluding the recipient check. Excluding a zero gives two ones among three messages; excluding a one gives one. Hence $x'_{v\to a} = 1 - s_a$ for these variables. A check with $s_a = 1$ is adjacent to p and receives exactly one one. A check with $s_a = 0$ is not adjacent to p and receives four ones, whose parity is zero. This is exactly s_a. The actual update equations are checked by `first_edge_matches`. □

Proof of the dynamical assertion of Theorem 1. The zero syndrome is accepted at initialization. Every nonzero syndrome is accepted after the first iteration by Lemma 2; Lemma 3 gives the same stopping decision for the alternate edge-parity convention. Extend a halted decoder by repeating its terminal output. All neighboring checks are then correct, and every variable is constant, for all times $t \geq 1$. This holds for every error e, not just the error with support A. There can therefore be no failure-inducing subset in Definition 3. The Lean theorem `absorbing_not_trapping` proves precisely these eventual properties. □

The extension by a constant terminal output represents the stated stopping algorithm. It does not claim that artificially continuing message updates after a successful stop would preserve every intermediate message.

4 An actual CSS counterexample with successful recovery

Syndrome acceptance alone does not guarantee quantum recovery. For example, using H in both sectors of the familiar Steane construction [3], errors on columns 1 and 2 produce the singleton output on column 3. The residual supported on $\{1, 2, 3\}$ has zero syndrome but is not in $\operatorname{row} H$. Thus it is a logical miscorrection. The source's dynamical Definition 3 and the broader notion of quantum decoding failure must be distinguished. We now eliminate this issue in a separate explicit CSS pair.

Let G consist of rows $1, 2, 4$ of H, and put

$$K = \begin{pmatrix} G \\ \mathbf{1} \end{pmatrix}, \qquad H_X = \begin{pmatrix} H & 0 \\ 0 & H \end{pmatrix}, \qquad H_Z = \begin{pmatrix} K & 0 \\ 0 & G \end{pmatrix}.$$

These are binary matrices of sizes 14×14 and 7×14. The redundant rows of H_X are retained as the actual decoding checks. For Z-errors, the decoder uses H_X and successful recovery means that the residual belongs to the *entire* row span of H_Z.

Lemma 4. $\ker H = \operatorname{row} K$, *and* $H_X H_Z^{\mathsf{T}} = 0$.

Proof. Every row of H has even weight, and two distinct rows have overlap two. Consequently $HG^{\mathsf{T}} = 0$ and $H\mathbf{1}^{\mathsf{T}} = 0$, so $\operatorname{row} K \subseteq \ker H$ and the displayed CSS matrices commute. For completeness, the reverse inclusion is checked on all 128 binary words against all 16 combinations of the four displayed generators. The Lean theorem `kernel_equals_four_generator_span` is this complete finite proof, not a search restricted by weight. The independent literal-matrix verifier obtains exactly the same kernel of 16 words. □

Proposition 5. *Let A be the first seven coordinates in this CSS pair. Every binary Z-error supported on A is decoded correctly modulo the complete Z-stabilizer span. There is nevertheless a nontrivial Z logical operator on the second block.*

Proof. There are no check edges between the two blocks. On input $(e, 0)$ the second block remains zero, while the first returns $(\hat{e}, 0)$ at initialization or after one iteration. By Lemma 2, $e+\hat{e} \in \ker H = \operatorname{row} K$. Hence $(e+\hat{e}, 0) \in \operatorname{row} H_Z$, so recovery is quantum-correct for all 128 supported errors. This exact complete-span membership is also checked directly by `first_block_quantum_recovery`.

The word $(0, \mathbf{1})$ commutes with every row of H_X, because all rows of H have even weight. It is not in $\operatorname{row} H_Z$: all second-block components in that span belong to $\operatorname{row} G$, whose words have even weight, whereas $\mathbf{1}$ has weight seven. This supplies a genuine nontrivial logical operator. Both statements are checked on the actual matrices by `logical_commutes` and `logical_not_stabilizer`. □

This construction explicitly uses a stabilizer-state component for the selected support, together with a component carrying logical information. It is not a claim that arbitrary high-weight errors on encoded information can be corrected. It shows why an unrestricted assertion about every absorbing set needs hypotheses on both the Tanner representation and the opposite stabilizer space.

5 Comparison with the source and earlier ingredients

The decoder here uses exactly zero initialization, extrinsic check XOR, extrinsic variable majority, zero ties, and syndrome stopping from Section II-B of [1]. Every possible subset of A is an input. There is no iteration cutoff, sampled failure estimate, or assumption excluding stabilizer errors. The first block has no exterior check neighbors, so the source convention that exterior messages are correct is automatically met. The checks and variables relevant to Definition 3 are all included.

The source proves the positive-b absorbing-set result in its Theorem 4; our example has $b = 0$ and does not contradict that theorem. Its path and cycle results concern other graphs; our matrix has degree four and is neither a path nor a cycle. In particular the counterexample does not imply that those cases, or useful high-girth code-design restrictions, are settled by this example.

Hamming's construction [2] already provides the syndrome-column representation used here. The simplex checks, the Steane code and the CSS stabilizer criterion are classical ingredients. The contribution here is their explicit application to the blanket absorbing-set question, the separation of dynamical trapping from logical miscorrection, and the complete checked counterexample. The two-block CSS example also refutes the stronger interpretation that some supported error must fail quantum recovery in the selected sector. It does not replace the opposite stabilizer span by the row span of the decoded matrix.

The directly inspected question is the May 10, 2024 arXiv version. A journal version appeared March 3, 2026 (DOI 10.3934/amc.2026023); its full text was access-restricted during the comparison. We therefore do not attribute unchanged wording to the journal version. The complete search and source-screening ledger is linked in Appendix A.

6 Conclusion

The full variable set of the redundant seven-column simplex matrix is absorbing, but none of its subsets traps the stated stopped Gallager-B algorithm. The explicit CSS pair additionally makes every supported error in the decoded sector quantum-correct. Thus an unrestricted implication from absorbing to trapping is false. Whether useful additional restrictions such as connected ambient CSS graphs, prescribed degree or girth conditions, or requiring the selected support to carry logical information restore such an implication is a separate question.

Source	Inspected milestone	Interval to October 26, 2026
Hamming	April 1950 publication	About 76.5 years (month precision)
Steane	January 29, 1996 preprint	11,228 days
Morris–Pllaha–Kelley	July 26, 2023 first preprint	1,188 days
Question in inspected version	May 10, 2024, v2, Section VI	899 days
Journal metadata	March 3, 2026 online publication	237 days
This paper	October 26, 2026	0 days

Table 1: Reference chronology. Ingredient age is not the duration for which an open question was unresolved. Exact day differences are independently regenerated in the artifact.

References

[1] K. D. Morris, T. Pllaha and C. A. Kelley, *Absorbing Sets in Quantum LDPC Codes*, arXiv:2307.14532v2, May 10, 2024, Sections II-B, III (Definitions 3 and 5), IV and VI. `https://arxiv.org/html/2307.14532v2`. Journal metadata: *Advances in Mathematics of Communications* 23 (2026), 234–254, `https://doi.org/10.3934/amc.2026023`.

[2] R. W. Hamming, *Error detecting and error correcting codes*, Bell System Technical Journal 29 (1950), 147–160. `https://doi.org/10.1002/j.1538-7305.1950.tb00463.x`.

[3] A. Steane, *Multiple particle interference and quantum error correction*, arXiv:quant-ph/9601029, January 29, 1996; Proceedings of the Royal Society A 452 (1996), 2551–2577. `https://arxiv.org/abs/quant-ph/9601029`.

A Complete checked proofs and source bridges

The single research module imports only `Std`. The proof toolchain is Lean 4.36.0-rc1, commit `c9afcddf514a6772bd4d649f7cfc04eff0b9f15e`. Every finite statement below is proved by kernel reduction with `decide`, not by a native decision oracle. The quantifier `e : BitVec 7` covers every one of the 128 subsets, including the empty and full supports. The clean build and axiom reports are included in the artifact. The permitted axioms are `propext`, `Classical.choice` and `Quot.sound`; no admitted proof is used.

A.1 Public sources and the proof map

The complete current Lean source and its immutable checked revision contain all research proofs, including the actual fourteen-coordinate bridge. The verification manifest, source and search ledger, and independent literal-matrix verifier make the input domain, access limits and evidence distinctions explicit.

Manuscript proof	Matching Lean declarations (all in the linked module)
Absorption and connectivity	`degrees`, `full_set_absorbing`, `first_block_connected`
Syndrome-column identity and Lemma 2	`syndrome_index_bridge`, `simplex_vote_counts`, `first_hard_formula`, `first_hard_matches`
Extrinsic update, Lemma 3	`first_edge_matches`, with the full `checkUpdate` and `variableUpdate` definitions

Manuscript proof	Matching Lean declarations (all in the linked module)
Theorem 1, eventual properties	`terminal_matches`, `all_eventually_correct`, `all_eventually_converge`, `absorbing_not_trapping`
Steane miscorrection comparison	`steane_logical_miscorrection`
Lemma 4	`kernel_equals_four_generator_span`, `actual_css_commutation`
Proposition 5, complete span and nontrivial logical	`first_block_residual_is_stabilizer`, `first_block_quantum_recovery`, `logical_commutes`, `logical_not_stabilizer`
Actual source matrices and fourteen-coordinate update	`actual_matrix_first_step_bridge`, `actual_matrix_syndrome_matches`, `terminal_eq_firstHard`, `actual_matrix_quantum_recovery`

A.2 Matrix, subsets and exact update rules

A seven-bit word is the indicator of a subset of the seven actual columns. The matrix is the binary dot product on labels $1, \ldots, 7$. `pack_bits` checks that packing and reading these column bits agree. The two message functions exclude the recipient explicitly, and the majority definition implements zero ties by a strict inequality. Here are the complete definitions and finite source checks. Unicode symbols are rendered typographically; the linked Lean file is the checked source.

```
import Std

/-! Exact seven-column simplex Tanner graph and source Gallager-B update.
All finite checks below are evaluated by Lean's kernel (‘decide‘), not native_decide.
The error type BitVec 7 ranges over all 128 subsets of the variable set. -/
namespace GallagerAbsorbing
set_option maxRecDepth 100000
set_option maxHeartbeats 0
set_option maxRecDepth 100000
abbrev Word := BitVec 7
abbrev Vertex := Fin 7
abbrev Messages := Vertex → Vertex → Bool

def parity (xs : List Bool) : Bool := xs.foldl Bool.xor false

def dot3 (a b : Nat) : Bool :=
  parity ((List.range 3).map fun j => a.testBit j && b.testBit j)

def H (c v : Vertex) : Bool := dot3 (c.val + 1) (v.val + 1)

def vertices : List Vertex := List.finRange 7

def bit (e : Word) (v : Vertex) : Bool := e.getLsbD v.val

def pack (f : Vertex → Bool) : Word :=
  BitVec.ofNat 7 ((vertices.map fun v => if f v then 2^v.val else 0).foldl (· + ·) 0)

def syndrome (e : Word) : Word :=
  pack fun c => parity (vertices.map fun v => H c v && bit e v)

def zeroMessages : Messages := fun _ _ => false

def checkUpdate (s : Word) (x : Messages) : Messages := fun c v =>
  H c v && (Bool.xor (bit s c)
    (parity (vertices.map fun u => H c u && (u != v) && x c u)))

def majority (xs : List Bool) : Bool := decide (xs.length < 2 * xs.countP id)

def hardDecision (y : Messages) : Word :=
```

```
  pack fun v => majority ((vertices.filter fun c => H c v).map fun c => y c v)

def variableUpdate (y : Messages) : Messages := fun c v =>
  H c v && majority ((vertices.filter fun d => H d v && (d != c)).map fun d => y d v)

def edgeSyndrome (x : Messages) : Word :=
  pack fun c => parity (vertices.map fun v => H c v && x c v)

def firstChecks (e : Word) := checkUpdate (syndrome e) zeroMessages

def firstHard (e : Word) := hardDecision (firstChecks e)

def firstExtrinsic (e : Word) := variableUpdate (firstChecks e)

def rowDegree (c : Vertex) := (vertices.filter fun v => H c v).length

def colDegree (v : Vertex) := (vertices.filter fun c => H c v).length

def absorbingFull : Prop :=
  ∀ v : Vertex,
    ((vertices.filter fun c => H c v && decide (rowDegree c % 2 = 1)).length <
     (vertices.filter fun c => H c v && decide (rowDegree c % 2 = 0)).length)

theorem degrees : (∀ c, rowDegree c = 4) ∧ (∀ v, colDegree v = 4) := by decide

theorem full_set_absorbing : absorbingFull := by
  unfold absorbingFull
  decide

theorem matrix_symmetric : ∀ c v, H c v = H v c := by decide

theorem pack_bits : ∀ e : Word, pack (bit e) = e := by decide

theorem first_hard_matches : ∀ e : Word, syndrome (firstHard e) = syndrome e := by decide

theorem first_edge_matches : ∀ e : Word,
    edgeSyndrome (firstExtrinsic e) = syndrome e := by decide

-- Self-orthogonality of the actual binary check rows, including identical rows.
def rowWord (c : Vertex) : Word := pack (H c)
def inner (e f : Word) : Bool := parity (vertices.map fun v => bit e v && bit f v)
theorem css_commutation : ∀ c d : Vertex, inner (rowWord c) (rowWord d) = false := by decide
```

A.3 From accepted output to absence of trapping

The dependency chain is `first_hard_matches` → `terminal_matches` → the two eventual properties → `absorbing_not_trapping`. The output at time zero is the initial zero guess. From time one onward it is the accepted terminal value. Thus the following unbounded-in-time proof is a structural argument, rather than a finite simulation horizon. The complete central proof bodies are printed here.

```
-- The stopped decoder is extended by its terminal output, as required to
-- interpret eventual convergence after an actual successful stopping time.
def terminal (e : Word) : Word := if syndrome e = 0 then 0 else firstHard e

def stoppedOutput (e : Word) (t : Nat) : Word := if t = 0 then 0 else terminal e

def eventuallyCorrect (e : Word) (c : Vertex) : Prop :=
  ∃ L : Nat, ∀ t : Nat, L ≤ t → bit (syndrome (stoppedOutput e t)) c = bit (syndrome e) c

def eventuallyConverges (e : Word) (v : Vertex) : Prop :=
  ∃ L : Nat, ∀ t : Nat, L ≤ t → bit (stoppedOutput e t) v = bit (stoppedOutput e (t+1)) v

def fullTrapping : Prop :=
```

```
  ∃ e : Word, (∃ c : Vertex, ¬ eventuallyCorrect e c) ∨
    (∃ v : Vertex, ¬ eventuallyConverges e v)

theorem terminal_matches : ∀ e : Word, syndrome (terminal e) = syndrome e := by decide

theorem all_eventually_correct (e : Word) (c : Vertex) : eventuallyCorrect e c := by
  refine ⟨1, ?_⟩
  intro t ht
  have h : t ≠ 0 := by omega
  simp only [stoppedOutput, ite_eq_right h]
  rw [terminal_matches]

theorem all_eventually_converge (e : Word) (v : Vertex) : eventuallyConverges e v := by
  refine ⟨1, ?_⟩
  intro t ht
  have h : t ≠ 0 := by omega
  have h' : t+1 ≠ 0 := by omega
  simp only [stoppedOutput, ite_eq_right h, ite_eq_right h']

theorem absorbing_not_trapping : absorbingFull ∧ ¬ fullTrapping := by
  constructor
  · exact full_set_absorbing
  · intro ⟨e, hf⟩
    rcases hf with ⟨c,hc⟩ | ⟨v,hv⟩
    · exact hc (all_eventually_correct e c)
    · exact hv (all_eventually_converge e v)

#print axioms absorbing_not_trapping
#print axioms first_hard_matches
#print axioms first_edge_matches
end GallagerAbsorbing
```

Direct declaration links: eventual check correctness, eventual variable convergence, and nontrapping theorem.

A.4 Full CSS span and logical witness

A pair of seven-bit words is exactly a binary word on the disjoint union of two seven-coordinate blocks. The matrices below retain all fourteen X checks and all seven Z generators. `combine` enumerates the entire generator span with independent binary coefficients; it does not check only individual stabilizer rows. The recovery statement covers every input and witnesses membership of the residual in that whole span. The nontrivial logical test uses the same matrix and span definitions.

```
namespace GallagerAbsorbing
-- Conventional CSS construction on fourteen actual binary coordinates, grouped
-- into two disjoint seven-coordinate blocks. Addition is componentwise XOR.
abbrev QWord := Word × Word

def qxor (a b : QWord) : QWord := (a.1 ^^^ b.1, a.2 ^^^ b.2)
def qinner (a b : QWord) : Bool := Bool.xor (inner a.1 b.1) (inner a.2 b.2)

def hxRows : List QWord :=
  (vertices.map fun c => (rowWord c, 0)) ++ (vertices.map fun c => (0, rowWord c))

def hzRows : List QWord :=
  [(rowWord 0, 0), (rowWord 1, 0), (rowWord 3, 0), (127, 0),
   (0, rowWord 0), (0, rowWord 1), (0, rowWord 3)]

def combine {n : Nat} (rs : List QWord) (a : BitVec n) : QWord :=
  (rs.zipIdx.map fun (r,i) => if a.getLsbD i then r else (0,0)).foldl qxor (0,0)

def hzSpan (w : QWord) : Prop := ∃ a : BitVec 7, combine hzRows a = w
```

```
theorem actual_css_commutation :
    hxRows.all (fun x => hzRows.all (fun z => !(qinner x z))) = true := by decide

theorem first_block_residual_is_stabilizer : ∀ e : Word,
    ∃ a : BitVec 7, combine hzRows a = (e ^^^ terminal e, 0) := by decide

theorem first_block_quantum_recovery (e : Word) :
    hzSpan (qxor (e,0) (terminal e,0)) := by
  simpa [hzSpan, qxor] using first_block_residual_is_stabilizer e

-- A nontrivial logical on the other block rules out a zero-logical-dimension
-- artifact. No distance claim is made or needed.
def logicalWitness : QWord := (0,127)

theorem logical_commutes :
    hxRows.all (fun x => !(qinner x logicalWitness)) = true := by decide

theorem logical_not_stabilizer : ¬ hzSpan logicalWitness := by
  unfold hzSpan
  decide

-- The component is disconnected from the spectator block, not from itself.
-- Every pair of first-block variables has a common neighboring check.
theorem first_block_connected : ∀ u v : Vertex,
    ∃ c : Vertex, H c u = true ∧ H c v = true := by decide

-- A concrete logical miscorrection in the ordinary Steane pairing H_X=H_Z=H.
def singleBlockSpan (e : Word) : Prop :=
  ∃ a : BitVec 3,
    (if a.getLsbD 0 then rowWord 0 else 0) ^^^
    (if a.getLsbD 1 then rowWord 1 else 0) ^^^
    (if a.getLsbD 2 then rowWord 3 else 0) = e

theorem steane_logical_miscorrection :
    terminal 3 = 4 ∧ syndrome 3 = syndrome 4 ∧ ¬ singleBlockSpan 7 := by
  unfold singleBlockSpan
  decide

#print axioms actual_css_commutation
#print axioms first_block_quantum_recovery
#print axioms logical_not_stabilizer
#print axioms first_block_connected
#print axioms steane_logical_miscorrection
end GallagerAbsorbing
```

Direct declaration links: complete residual-span theorem and nontrivial logical theorem.

A.5 Analytic identities and the actual fourteen-coordinate decoder

The finite formulas below supply the syndrome-column, vote-count and kernel/span identities used in the written proofs. The final portion then calculates the syndrome and all first-step votes directly from the actual fourteen-coordinate rows. Its bridge theorem proves that this calculation is the component calculation used above. This avoids assuming that an abstract block argument automatically matches the source matrix.

```
namespace GallagerAbsorbing
set_option maxHeartbeats 0
set_option maxRecDepth 100000

def syndromeIndex (e : Word) : Nat :=
  (vertices.map fun v => if bit e v then v.val+1 else 0).foldl Nat.xor 0

def votes (p : Fin 8) (v : Vertex) : Nat :=
  (vertices.filter fun c => H c v && dot3 (c.val+1) p.val).length
```

```
theorem syndrome_index_bridge : ∀ e : Word, ∀ c : Vertex,
    bit (syndrome e) c = dot3 (c.val+1) (syndromeIndex e) := by decide

theorem simplex_vote_counts : ∀ p : Fin 8, ∀ v : Vertex,
    votes p v = if p.val = 0 then 0 else if p.val = v.val+1 then 4 else 2 := by decide

theorem first_hard_formula : ∀ e : Word,
    firstHard e = if syndromeIndex e = 0 then 0 else
      BitVec.ofNat 7 (2^(syndromeIndex e - 1)) := by decide

def kernelCombination (a : BitVec 4) : Word :=
  (if a.getLsbD 0 then rowWord 0 else 0) ^^^
  (if a.getLsbD 1 then rowWord 1 else 0) ^^^
  (if a.getLsbD 2 then rowWord 3 else 0) ^^^
  (if a.getLsbD 3 then 127 else 0)

theorem kernel_equals_four_generator_span : ∀ e : Word,
    syndrome e = 0 ↔ ∃ a : BitVec 4, kernelCombination a = e := by decide

#print axioms first_hard_formula
#print axioms kernel_equals_four_generator_span
end GallagerAbsorbing

namespace GallagerAbsorbing
set_option maxHeartbeats 0
set_option maxRecDepth 100000
abbrev QVertex := Fin 14

def qbit (w : QWord) (v : QVertex) : Bool :=
  if v.val < 7 then w.1.getLsbD v.val else w.2.getLsbD (v.val-7)

def hxRow (c : QVertex) : QWord := hxRows[c.val]!

def actualSyndrome14 (w : QWord) : BitVec 14 :=
  BitVec.ofNat 14 (((List.finRange 14).map fun c =>
    if qinner (hxRow c) w then 2^c.val else 0).foldl (· + ·) 0)

def actualFirstChecks14 (e : QWord) (c v : QVertex) : Bool :=
  qbit (hxRow c) v && (actualSyndrome14 e).getLsbD c.val

def actualFirstVote14 (e : QWord) (v : QVertex) : Bool :=
  majority (((List.finRange 14).filter fun c => qbit (hxRow c) v).map
    fun c => actualFirstChecks14 e c v)

def actualFirstHard14 (e : QWord) : QWord :=
  (pack fun v => actualFirstVote14 e ⟨v.val, by omega⟩,
   pack fun v => actualFirstVote14 e ⟨v.val+7, by omega⟩)

theorem actual_matrix_first_step_bridge : ∀ e : Word,
    actualFirstHard14 (e,0) = (firstHard e,0) := by decide

theorem actual_matrix_syndrome_matches : ∀ e : Word,
    actualSyndrome14 (actualFirstHard14 (e,0)) = actualSyndrome14 (e,0) := by decide

theorem terminal_eq_firstHard : ∀ e : Word, terminal e = firstHard e := by decide

theorem actual_matrix_quantum_recovery (e : Word) :
    hzSpan (qxor (e,0) (actualFirstHard14 (e,0))) := by
  rw [actual_matrix_first_step_bridge, ← terminal_eq_firstHard]
  exact first_block_quantum_recovery e

#print axioms actual_matrix_first_step_bridge
#print axioms actual_matrix_syndrome_matches
#print axioms actual_matrix_quantum_recovery
end GallagerAbsorbing
```

Direct declaration link: actual-matrix quantum recovery theorem.

A.6 Reproduction and evidence limits

Run `python3 build.py -lean /path/to/lean` from the artifact folder with the pinned toolchain, then `python3 verify.py -check`. The Lean result is the proof; the independently regenerated JSON certificate provides a second, literal-matrix reconstruction of all inputs. No sampled experiment is used. The certificate also supplies independent linear-algebra diagnostics; numerical diagnostics not stated as theorems in the manuscript are not additional formal results. The source ledger distinguishes the accessible arXiv source, the restricted journal text, partial classical-source retrievals and historical questions harvested without claiming they remain open today.